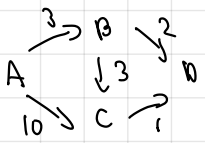
lecture 16
dynamic programming on graphs



Single Source Shortest Path:
starting at node A, what is the shortest path for every other node

Path: sequence of nodes connected by edges
Path weight: sum of edges in path
Shortest Path: weight of path with smallest weight

Shortest paths decompose to other shortest path

Suppose path from u to v
then all of the prefixes are also shortest paths

u → ... w → v
w is the step before v
u → v is also a shortest path

If w is on the way
u → w is also the shortest path

Generally:

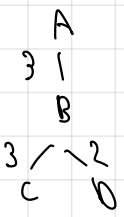
$$\text{shortest_path}(u \to v) = \min_{w} \; \text{shortest_path}(u, w) + w \to v$$

Output: To get the shortest path to a node, what is the next hop
good representation: save space

From A

Node	Pred	Dist
A	—	0
B	A	3
C	B	6
D	B	5

A → B → D
A → B → C



Can make shortest path tree

Recon D → B → A

*in our table, we just need to store the last node in the shortest path

---

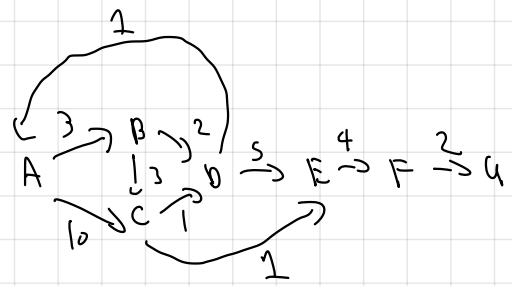
Negative weight cycle

- The path will just be endless loops of the negative weight cycle
- can get around with no duplicate edges or nodes, but that is away from the goal

Assume no negative weight cycles

Node	Pred	Dist
A	—	0
B	A	3
C	B	6
D	B	5
E	C	7
F	E	11
G	F	13

A → B
A → B → C
A → B → D
A → B → C → E
A → B → C → E → F
A → B → C → E → F → G



Relax operation

Both algorithms fill out tables, initially over approximations (∞)
- initially assume no path

Loop through all edges and apply relax operation

```
def relax(u, dst, pred, edge):
    if dst[edge.dst] > dst[edge.src] + edge.weight:
        dst[edge.dst] ← dst[edge.src] + edge.weight
        pred[edge.dst] ← edge.src
```

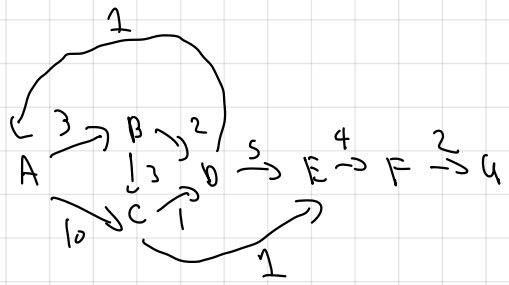
given:
node u
edge $w \xrightarrow{s} v$

On all edges: is this edge the final edge in the path from $u \to v$
starting at node u
given edge $w \to v$ with weight w

$dst[edge.src] + w$ $u \longrightarrow w \to v$	is there a new dist for $u \to v$?

edge $w \to v$
src: w
dst: v

Node	Pred	Dist
A	—	0
B	A	~~∞~~ 3
C	B	~~∞~~ 6
D		∞
E		∞
F		∞
G		∞



relax $A \xrightarrow{3} B$ check if distance to A in table + A→B edge weight 3 is less than distance to B
edge
src: A
dst: B
weight: 3
$0 + 3 < \infty$

B A
$dst[edge.dst] = \infty > dst[edge.src] + edge.weight = 0 + 3$
$dst[edge.dst] = 0 + 3$
$pred[edge.dst] \leftarrow A$

---

relax $B \xrightarrow{3} C$ update $\infty \to 3 + 3$ use this path ✓

relax $A \xrightarrow{10} C$ no update ~~6~~ 10. don't use this path

---

Bellman Ford Algorithm

```
def bellman_ford(graph, start):

    dist = {node: infinity for node in graph}
    dist[start] = 0
    pred = {node: none for node in graph}

    for i = 1, 2, ... n:
        for edge in graph.edges:
            relax(graph, dist, pred, edge)
        if nothing changed:
            break

    return dist, pred

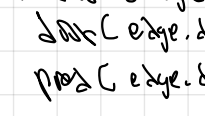
def relax(graph, dist, pred, edge):
    if dist[edge.dst] > dist[edge.src] + edge.weight:
        dist[edge.dst] = dist[edge.src] + edge.weight
        pred[edge.dst] = edge.src
```

setting up: O(n)

loop through edges relax edges

until nothing changes

O(n·m)



node	pred	dist	visited?
A	–	0	✓
B	A	~~∞~~ 1	✓
C	B	~~∞~~ 2	✓
D	A C	~~∞~~ 10 3	✓
E	D	~~∞~~ 4	✓

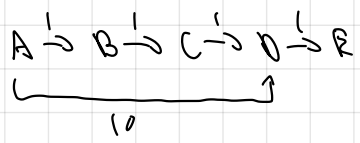
iteration:	relax
1 A	A→B, A→D
2 B	B→C
3 C	C→D
4 D	D→E
5 E	

Dynamically figures out best way to visit nodes
- only visit a node when you have a shorter path to it

There can't be negative weights since the greedy algorithm assumes we have shortest paths to nodes dynamically

recall bellman ford can't have negative cycles

Why n iterations:
- all shortest paths don't have cycles, since those would cost more weights
- only makes sense to have cycles if they are negative weight cycles
  - recall there are no negative cycles
- since there are n nodes, must end in n iterations
- acyclic means < n steps



iterate through all edges

Best:

Node	Pred	Dist
A	–	0
B	A	~~∞~~ 1
C	B	~~∞~~ 2
D	C	~~∞~~ 3
E	D	~~∞~~ 4

A→B	1
B→C	1
C→D	1
D→E	1
A→D	10

Worst:

Node	Pred	Dist
A	–	0
B	A	~~∞~~ 1
C	B	~~∞~~ 2
D	~~A~~ C	~~∞~~ ~~10~~ 3
E	~~D~~ D	~~∞~~ ~~10~~ 4

D→E	1
C→D	1
B→C	1
A→D	10
A→B	1

after 1 iteration, 1 step paths from A
after 2 iterations, 2 step paths from A
⋮
after n iterations, n step paths from A

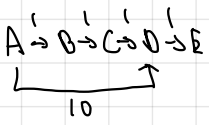
Best
- 2 iterations
- O(n+m)

Worst:
- n iterations
- O(n·m + n)

Dijkstra's Algorithm

Figure out order to relax edges: relax smaller distance

- out of all unvisited nodes, visit the one with the smaller distance
- when you visit a node, relax all outgoing edges



node	pred	dist	visited?
A	–	0	✓
B	A	~~∞~~ 1	✓
C	B	~~∞~~ 2	✓
D	A C	~~∞~~ ~~10~~ 3	✓
E	D	~~∞~~ 4	✓

iteration:	relax
1 A	A→B, A→D
2 B	B→C
3 C	C→D
4 D	D→E
5 E	

```
def dijkstra(graph, start):

    dist = {node: infinity for node in graph}
    dist[start] = 0
    pred = {node: none for node in graph}

    not_visited = set(graph.nodes)          # heap
    while not_visited:                      # O(n) n nodes
        # return element with smallest dist[node] value
        node = not_visited.pop_smallest_dist()   # pop on heap O(log n)
        for edge in graph.Outgoing[node]:        # in total, total O(log n) to update heap
            relax(graph, dist, pred, edge)       # log(n) per edge called
    return dist, pred
```

O(n·log(n) + m·log(n))

O((n+m) log(n))

Unweighted graph:	Weighted graph:
- use a list/queue	- use a min heap
- O(1) insertion	- O(log n) insertion
O(n+m)	O((n+m) log(n))

Breadth first search Dijkstra

---

run time
- use a heap to find the node of smallest distance. O(log(n)) time search